

[47] Product Quantization for Nearest Neighbor Search

요약

본 논문은 Hervé Jégou, Matthijs Douze, Cordelia Schmid에 의해 2011년 IEEE TPAMI에 발표된 매우 영향력 있는 작업이다. Product Quantization (PQ)은 고차원 벡터를 저차원 양자화 코드로 압축하는 기법으로, 대규모 벡터 검색 시스템에서 메모리 효율성과 검색 속도를 획기적으로 개선한다.

PQ의 핵심 아이디어는 고차원 벡터를 여러 개의 저차원 부분공간(subspace)으로 분할하고, 각 부분공간에 대해 독립적으로 양자화를 수행하는 것이다. 예를 들어, 128차원 벡터를 4개의 32차원 부분공간으로 분할하고, 각 부분공간에서 256개의 양자화 중심(centroid)을 학습한다. 그 결과 각 벡터를 4개의 바이트(또는 코드워드)로 표현할 수 있으며, 원본 벡터 대비 32배의 압축률을 달성한다.

검색 과정에서는 쿼리 벡터도 동일하게 양자화되고, 벡터 간의 거리는 양자화된 코드 간의 거리로 근사된다. PQ는 이러한 거리 계산을 매우 효율적으로 수행할 수 있도록 설계되어 있다. 각 부분공간에서 쿼리 벡터와 모든 중심 간의 거리를 미리 계산하여 테이블을 만든 후, 데이터베이스의 각 벡터에 대해서는 테이블 룩업만으로 거리를 계산할 수 있다.

PQ는 다양한 벡터 차원의 데이터셋에 적용 가능하며, 실제로 SIFT(Scale-Invariant Feature Transform) 디스크립터 같은 고정 차원 특징 벡터를 다루는 컴퓨터 비전 애플리케이션에서 광범위하게 사용되었다. 또한 역벡터 인덱싱(inverted vector indexing)과 결합되어 대규모 벡터 검색 시스템의 성능을 크게 향상시켰다.

상세분석

47.1 Product Quantization의 원리

Product Quantization은 고차원 벡터의 유사성 검색 문제를 효율적으로 해결하기 위해 설계된 기법이다. 기본 아이디어는 'product'라는 이름에서도 알 수 있듯이, 전체 공간을 여러 개의 저차원 부분공간의 곱(product)으로 분해하는 것이다.

구체적인 프로세스는 다음과 같다. 먼저, d 차원 벡터를 m 개의 d/m 차원 부분벡터로 분할한다. 예를 들어 $d=128$, $m=4$ 인 경우, 각 부분벡터는 32차원이다. 각 부분공간에 대해 독립적인 양자화(quantization)를 수행한다. 이는 k -means 클러스터링을 사용하여 각 부분공간에서 k 개의 클러스터 중심(양자화 중심)을 학습하는 것을 의미한다.

양자화 중심의 개수 k 는 각 부분벡터를 몇 비트로 인코딩할 것인지에 따라 결정된다. 일반적으로 $k=256$ (8비트)을 사용하므로, 전체 벡터는 m 개의 8비트 코드의 조합, 즉 8m비트로 표현된다. 128차원을 8비트로 압축하면 약 16배 압축이 되며, 원본 벡터 (128×4 바이트 = 512바이트) 대비 4바이트만으로 표현할 수 있다.

각 부분공간에서의 양자화는 다음과 같이 진행된다. 부분공간 j 에 속한 모든 벡터들의 부분벡터들을 추출하고, 이에 대해 k -means 알고리즘을 실행한다. 결과적으로 k 개의 중심 벡터 $c_{j,0}, c_{j,1}, \dots, c_{j,k-1}$ 이 학습된다. 각 훈련 벡터의 부분벡터는 가장 가까운 중심에 할당되고, 그 중심의 인덱스를 코드로 저장한다.

47.2 거리 계산의 효율성

PQ의 가장 중요한 성능 이점은 거리 계산의 효율성에 있다. 전통적인 방식으로는 n 개의 d 차원 벡터와 1개의 쿼리 벡터 간의 거리를 모두 계산하는 데 $O(n \cdot d)$ 의 시간이 필요하다. PQ는 이를 훨씬 더 빠르게 수행할 수 있다.

PQ의 거리 계산 방식은 다음과 같다. 먼저 쿼리 벡터 q 에 대해, 각 부분공간에서 q 의 부분벡터와 모든 양자화 중심 간의 거리를 미리 계산한다. 부분공간 j 에서는 k 개의 거리 값이 생성되므로, m 개 부분공간에 대해 총 mk 개의 거리를 미리 계산한다. 이는 $O(mkd/m) = O(kd)$ 시간에 수행된다.

다음으로, 데이터베이스의 각 벡터 x 에 대해, x 의 양자화 코드($code_0, code_1, \dots, code_{m-1}$)를 사용하여 거리를 계산한다. 부분공간 j 에서 $code_j$ 번째 미리 계산된 거리 값을 조회하면 되므로, 각 벡터당 단 m 번의 테이블 룩업만으로 거리를 계산할 수 있다. 따라서 총 시간은 $O(nm)$ 이 되며, 이는 $O(nd)$ 에 비해 훨씬 빠르다 ($d \gg k$ 인 경우).

47.3 손실과 정확도

PQ는 손실 압축(lossy compression)이므로, 당연히 정보 손실이 발생한다. 그러나 유사성 검색의 맥락에서는 이러한 손실이 심각한 문제가 되지 않을 수 있다. 왜냐하면 우리가 관심 있는 것은 정확한 거리 값이 아니라 상대적인 거리 순서이기 때문이다.

PQ의 정확도는 여러 요인에 의해 영향을 받는다. 첫째, 부분공간의 개수 m 이다. m 이 작을수록 압축률은 높지만 정확도는 낮다. 일반적으로 $m=4 \sim 8$ 정도가 좋은 균형을 제공한다.

둘째, 각 부분공간의 양자화 중심 개수 k 이다. k 가 작을수록 압축률은 높지만 정확도는 낮다. $k=256$ (8비트)은 많은 응용에서 충분히 높은 정확도를 제공한다.

셋째, 학습 데이터와 쿼리 데이터의 분포의 일치도이다. PQ 모델이 학습한 분포와 실제 쿼리 분포가 다르면 정확도가 저하될 수 있다.

실제 실험에서 PQ는 다음과 같은 특성을 보여준다. SIFT 1M 데이터셋에서 99%의 상대적 정확도(recall@1000)를 달성하면서 압축률은 약 32배를 제공한다. 또한 거리 계산 속도는 원본 벡터 사용 시보다 수십 배 빠르다.

47.4 역벡터 인덱싱과의 결합

PQ는 역벡터 인덱싱(inverted vector index, IVF)과 결합되어 더욱 강력한 검색 시스템을 구성할 수 있다. IVF는 벡터를 먼저 대략적으로 클러스터링한 후, 쿼리와 유사한 클러스터만을 검사하는 방식이다.

IVF+PQ 조합에서는 다음 과정이 진행된다. 먼저 모든 벡터를 대략 k개 클러스터로 분할한다 (보통 $k=100\sim10000$). 그 다음, 각 벡터를 PQ로 압축한다. 검색 시에는 쿼리 벡터가 속한 클러스터의 인근 클러스터들만 검사하고, 각 클러스터 내에서는 PQ 기반의 거리 계산을 사용한다.

이러한 조합은 다음과 같은 장점을 제공한다. 검사 대상이 되는 벡터의 개수를 크게 줄일 수 있으므로, 전체 검색 시간은 크게 단축된다. 또한 메모리 사용량도 PQ로 인해 크게 감소한다. 예를 들어, IVF+PQ 조합으로 1억 개의 벡터를 검색하면서도 메모리 사용량을 GB 단위로 유지할 수 있다.

47.5 재분석 및 변형들

PQ 이후로 다양한 변형이 제안되었다. OPQ (Optimized Product Quantization)는 PQ의 부분공간 분할을 더 최적화하는 방법이다. 각 부분공간의 분산을 최대화하도록 벡터를 회전시킨 후 PQ를 적용하면 정확도를 더욱 향상시킬 수 있다.

또 다른 중요한 변형은 Composite Quantization (CQ)이다. CQ는 여러 양자화 방법을 결합하는 접근 방식으로, PQ의 부분공간 독립성 가정을 완화한다. 이를 통해 부분공간 간의 상관관계를 활용하여 더 높은 정확도를 달성할 수 있다.

47.6 본 논문과의 관계

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. 카디널리티 추정은 쿼리 최적화기(query optimizer)가 가장 효율적인 실행 계획을 선택하기 위해 필요한 정보다. PQ는 유사성 검색 인덱스의 중요한 구성 요소이며, PQ 기반 검색의 결과 크기(즉, 카디널리티)를 정확하게 예측하려면 특별한 추정 기법이 필요하다.

특히, PQ는 손실 압축이므로 거리가 변형된다. 따라서 특정 거리 임계값 이내의 벡터를 검색할 때 결과의 개수를 예측하는 것이 매우 어렵다. Exqutor가 제시하는 머신러닝 기반의 카디널리티 추정 기법은 PQ와 같은 압축 방법을 사용하는 벡터 검색 시스템에서 정확한 카디널리티 추정을 가능하게 한다.

추가 제기 문제

1. **부분공간 분할의 최적화**: 부분공간을 독립적으로 분할할 때, 최적의 분할 방식에 대한 이론적 지침이 부족하다. 데이터 특성에 따라 최적의 m 값을 선택하는 방법이 명확하지 않다.
2. **동적 데이터에의 적용**: PQ 모델이 학습된 후 새로운 벡터가 추가될 때, 모델을 재학습해야 하는지 아니면 기존 중심을 사용할 수 있는지에 대한 명확한 지침이 없다.
3. **카디널리티 추정의 어려움**: PQ 압축으로 인한 거리 변형이 유사성 쿼리의 결과 크기 예측에 미치는 영향을 정량화하기 어렵다. 이것이 Exqutor 같은 학습 기반 추정 방법이 필요한 이유이다.
4. **고차원의 한계**: 벡터의 차원이 매우 높아지면, 부분공간도 많아져야 하고, 이에 따라 계산 오버헤드가 증가한다.